BUSINESS INTELLIGENCE AND DATA ANALYTICS

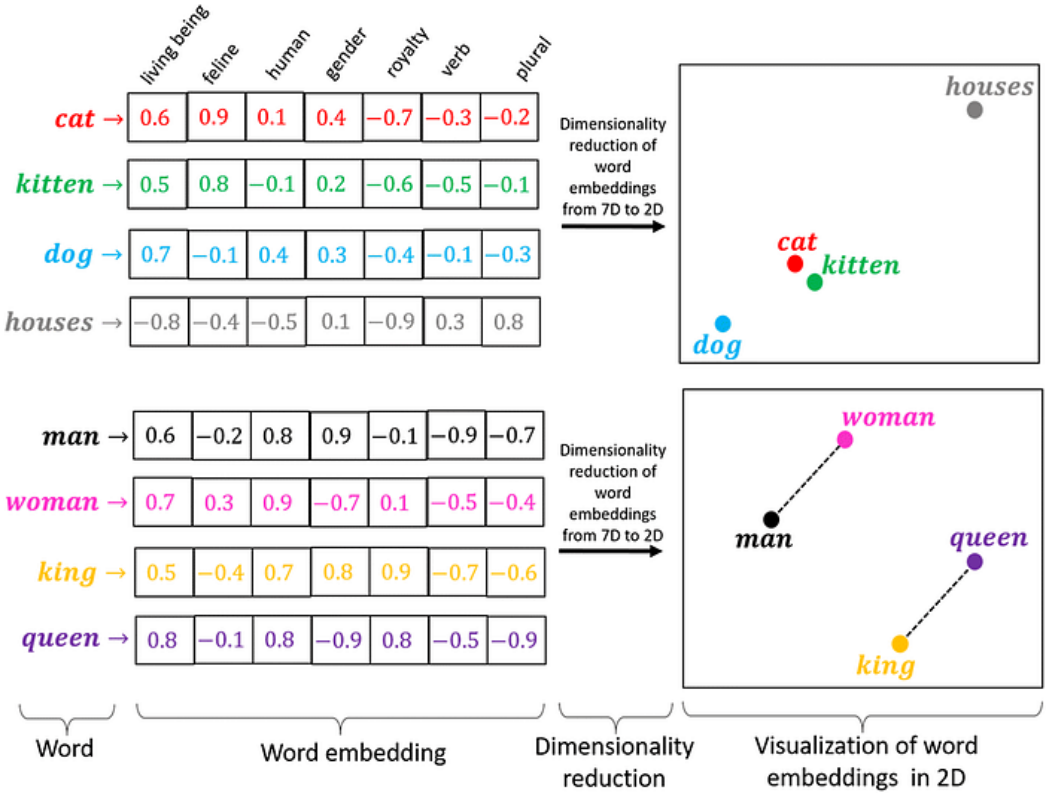
University of Petra

Institute of Analytics
Recognized Programme

307307
Introduction to Neural Networks and Word Embeddings

Preface

- In the previous section, we used Bow and TDIDF to convert documents and words into numerical representations.
- These representations were simple, they had no semantics for words, just on/off switches for existing and non-existing words in a document.
- In this part of the course, we want to convert words into meaningful list of numbers.
- These numbers are called **Word Embeddings**.
- We will use Neural Networks to create these Word Embeddings.



History of Neural Networks

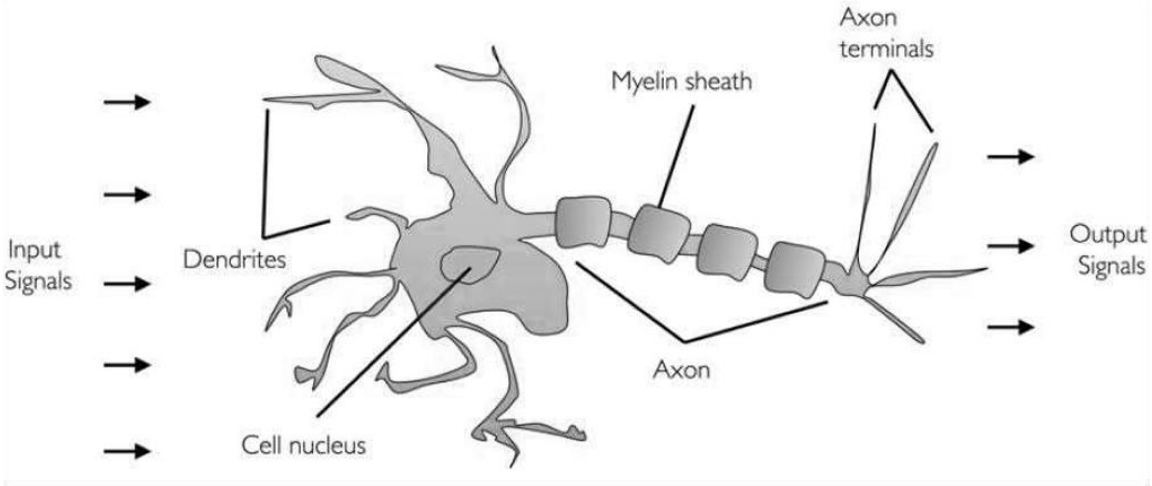
- In 1943, researchers Warren McCullock and published their first concept of simplified brain cell.
- This was called McCullock-Pitts (MCP) neuron.
- They described such a nerve cell as a simple logic gate with binary outputs.
- Multiple signals arrive at the dendrites and are then integrated into the cell body, and, if the accumulated signal exceeds a certain threshold, an output signal is generated that will be passed on by the axon.

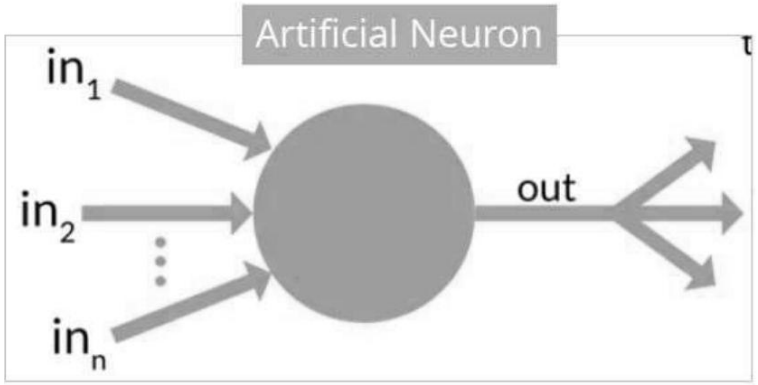
Warren Sturgis McCulloch

(1898 – 1969)

Walter Harry Pitts, Jr.

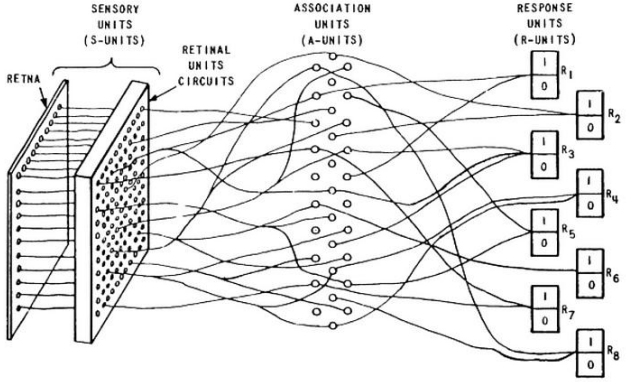
(1923 – 1969)



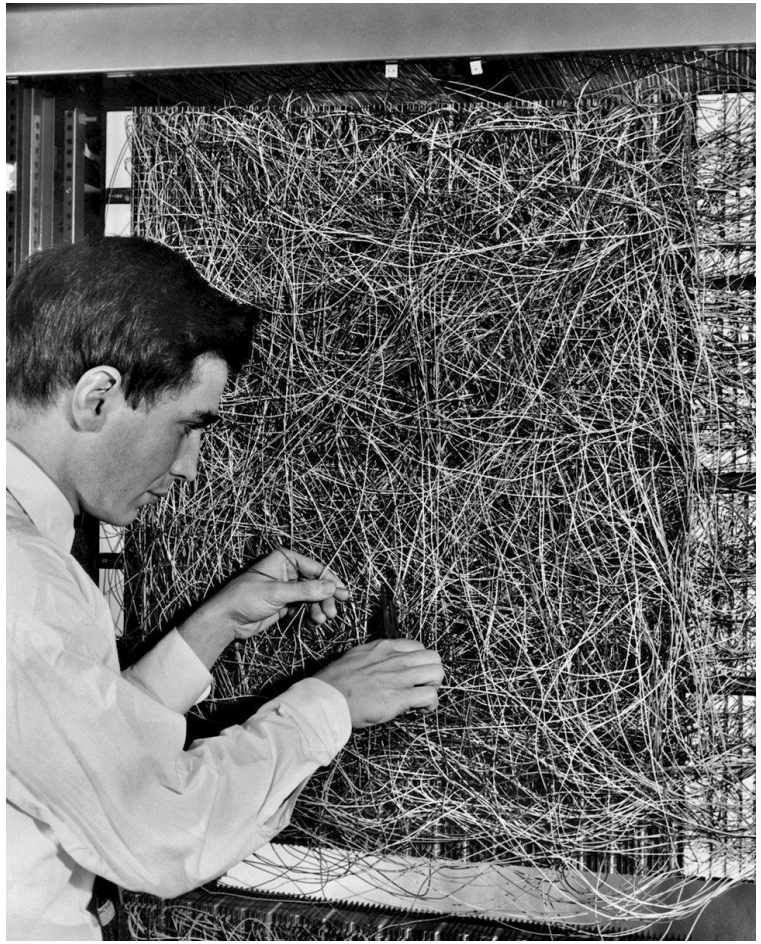


The Perceptron: Building Block of Neural Networks

- In 1953, inspired by McCullock work, Frank Rosenblatt invented the Perceptron.
- The Perceptron is the simplest form of a neural network
- Binary classifier: separates data into two categories
- Models a single neuron with multiple inputs and one output

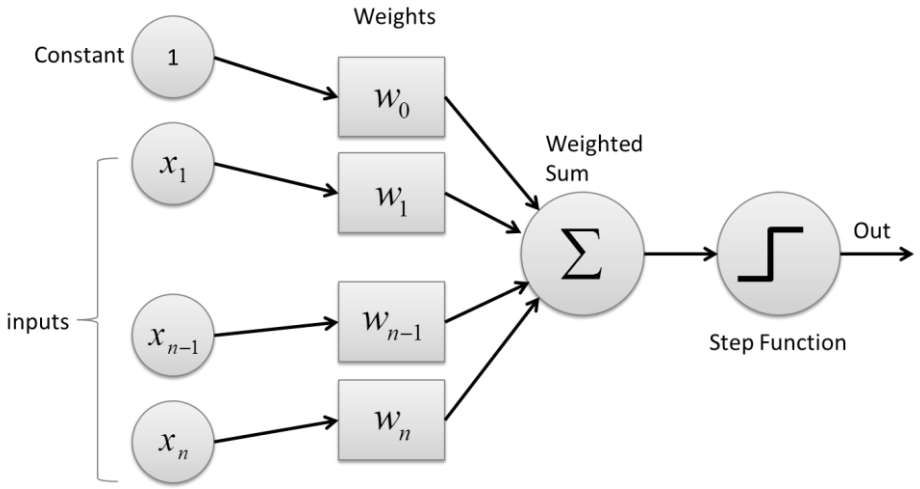


F. Rosenblatt



The Perceptron

- Inputs: $x_1, x_2, \ldots, x_n$
- Weights: $w_1, w_2, \ldots, w_n$
- Bias: b
- Activation function: Step function
- Output: 1 if weighted sum > threshold, 0 otherwise

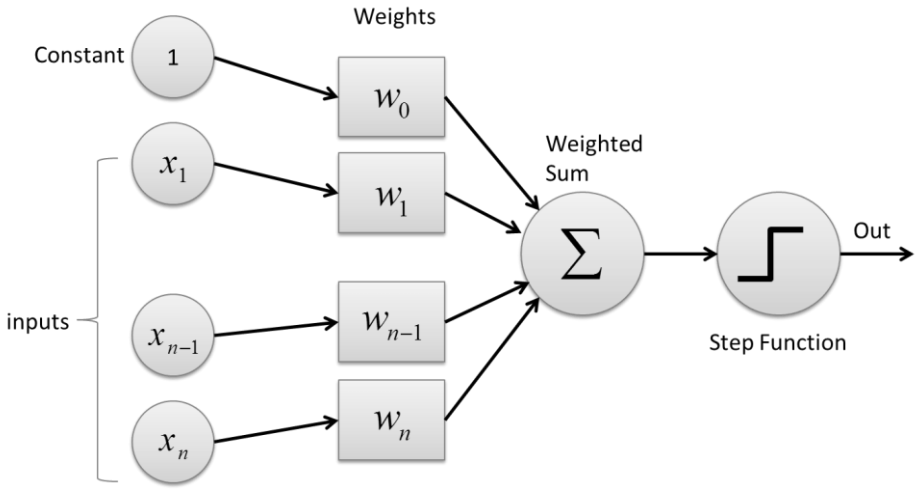


How a Perceptron Works

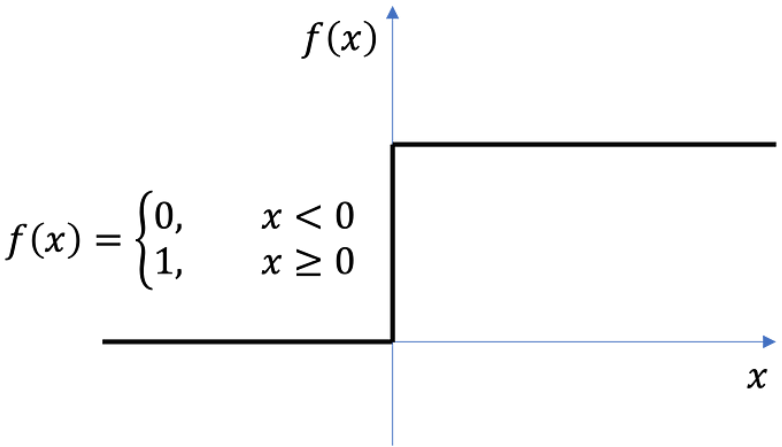
1. Multiply each input by its corresponding weight
2. Sum all weighted inputs
3. Add the bias term
4. Apply the activation function
5. Output the result

Mathematically:

- $z = w_1x_1 + w_2x_2 + \ldots + w_nx_n + b$
- output = activation(z)



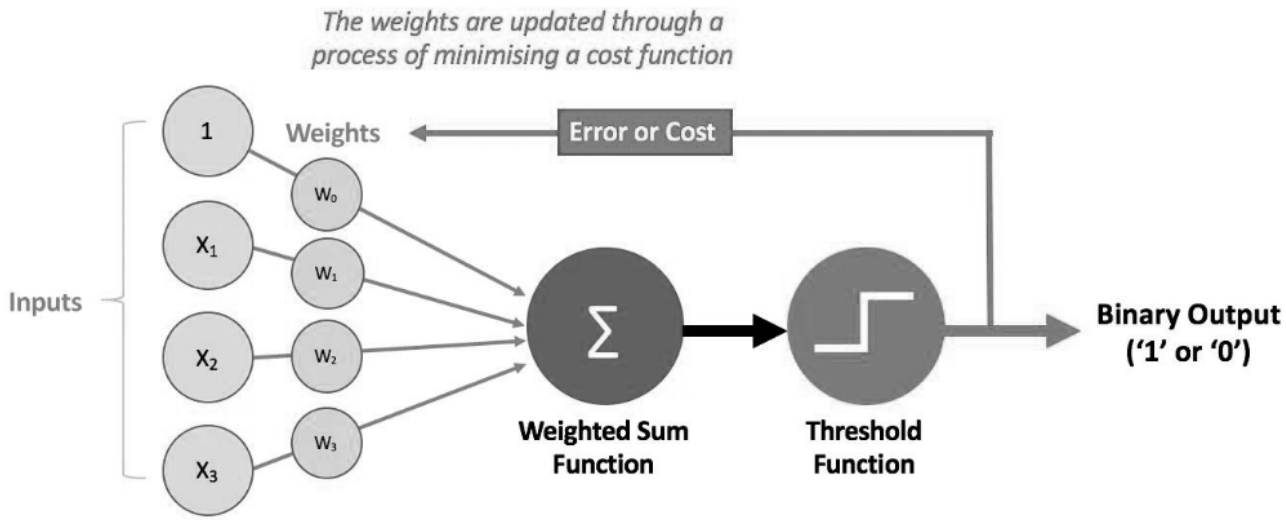
- **Step Function**:
  - Output: 1 if z ≥ 0, 0 if z < 0
  - Used in original perceptrons
  - Not differentiable at 0



How Perceptron Learn (The Cost Function)

For each training example:

1. Calculate predicted output y_pred
2. Calculate error: error = y_true - y_pred
3. Update weights: w_new = w_old + learning_rate * error * x
4. Update bias: b_new = b_old + learning_rate * error



Step-by-Step Hand Calculation for AND Gate

Let's work through the perceptron learning algorithm by hand for the AND gate:

- Training data: X = [[0,0], [0,1], [1,0], [1,1]], y = [0, 0, 0, 1]
- Learning rate (η) = 0.1
- Initial weights (randomly assigned): $w_1 = 0.3$, $w_2 = -0.1$
- Initial bias: b = 0.2

First Iteration:

Example 1: (0,0) → 0

- Inputs: $x_1 = 0$, $x_2 = 0$
- Weighted sum: $z = w_1x_1 + w_2x_2 + b = 0.3(0) + (-0.1)(0) + 0.2 = 0.2$
- Activation: output = 1 (since $z > 0$)
- True output: y = 0
- Error: error = y - output = 0 - 1 = -1
- Weight updates:
  - $w_1 = w_1 + \eta * \text{error} * x_1 = 0.3 + 0.1 * (-1) * 0 = 0.3$
  - $w_2 = w_2 + \eta * \text{error} * x_2 = -0.1 + 0.1 * (-1) * 0 = -0.1$
  - $b = b + \eta * \text{error} = 0.2 + 0.1 * (-1) = 0.1$

Step-by-Step Hand Calculation for AND Gate

Example 2: (0,1) → 0

- Inputs: $x_1 = 0$, $x_2 = 1$
- Weighted sum: $z = w_1x_1 + w_2x_2 + b = 0.3(0) + (-0.1)(1) + 0.1 = 0$
- Activation: output = 1 (since $z \geq 0$)
- True output: $y = 0$
- Error: error = y - output = 0 - 1 = -1
- Weight updates:
  - $w_1 = w_1 + \eta * \text{error} * x_1 = 0.3 + 0.1 * (-1) * 0 = 0.3$
  - $w_2 = w_2 + \eta * \text{error} * x_2 = -0.1 + 0.1 * (-1) * 1 = -0.2$
  - $b = b + \eta * \text{error} = 0.1 + 0.1 * (-1) = 0$

Example 3: (1,0) → 0

- Inputs: $x_1 = 1$, $x_2 = 0$
- Weighted sum: $z = w_1x_1 + w_2x_2 + b = 0.3(1) + (-0.2)(0) + 0 = 0.3$
- Activation: output = 1 (since $z > 0$)
- True output: $y = 0$
- Error: error = y - output = 0 - 1 = -1
- Weight updates:
  - $w_1 = w_1 + \eta * \text{error} * x_1 = 0.3 + 0.1 * (-1) * 1 = 0.2$
  - $w_2 = w_2 + \eta * \text{error} * x_2 = -0.2 + 0.1 * (-1) * 0 = -0.2$
  - $b = b + \eta * \text{error} = 0 + 0.1 * (-1) = -0.1$

Step-by-Step Hand Calculation for AND Gate

Example 4: (1,1) → 1

- Inputs: $x_1 = 1$, $x_2 = 1$
- Weighted sum: $z = w_1x_1 + w_2x_2 + b = 0.2(1) + (-0.2)(1) + (-0.1) = -0.1$
- Activation: output = 0 (since $z < 0$)
- True output: $y = 1$
- Error: error = y - output = 1 - 0 = 1
- Weight updates:
  - $w_1 = w_1 + \eta * \text{error} * x_1 = 0.2 + 0.1 * 1 * 1 = 0.3$
  - $w_2 = w_2 + \eta * \text{error} * x_2 = -0.2 + 0.1 * 1 * 1 = -0.1$
  - $b = b + \eta * \text{error} = -0.1 + 0.1 * 1 = 0$

End of Iteration 1:

- Updated weights: $w_1 = 0.3$, $w_2 = -0.1$
- Updated bias: $b = 0$

Second Iteration

Example 1: (0,0) → 0

- Inputs: $x_1 = 0$, $x_2 = 0$
- Weighted sum: $z = w_1x_1 + w_2x_2 + b = 0.3(0) + (-0.1)(0) + 0 = 0$
- Activation: output = 1 (since $z \geq 0$)
- True output: $y = 0$
- Error: error = y - output = 0 - 1 = -1
- Weight updates:
  - $w_1 = w_1 + \eta * \text{error} * x_1 = 0.3 + 0.1 * (-1) * 0 = 0.3$
  - $w_2 = w_2 + \eta * \text{error} * x_2 = -0.1 + 0.1 * (-1) * 0 = -0.1$
  - $b = b + \eta * \text{error} = 0 + 0.1 * (-1) = -0.1$

Example 2: (0,1) → 0

- Inputs: $x_1 = 0$, $x_2 = 1$
- Weighted sum: $z = w_1x_1 + w_2x_2 + b = 0.3(0) + (-0.1)(1) + (-0.1) = -0.2$
- Activation: output = 0 (since $z < 0$)
- True output: $y = 0$
- Error: error = y - output = 0 - 0 = 0
- Weight updates (no change as error = 0):
  - $w_1 = 0.3$
  - $w_2 = -0.1$
  - $b = -0.1$
- **After several iterations**, the perceptron will converge to weights that correctly classify all AND gate examples.

Python Implementation Perceptron from Scratch

```python
from sklearn.linear_model import Perceptron
import numpy as np

   # Training data for AND gate
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([0, 0, 0, 1])

   # Initialize and train Perceptron
model = Perceptron(max_iter=100, eta0=0.1, random_state=42)
model.fit(X, y)
# Results
print("Weights:", model.coef_)
print("Bias:", model.intercept_)
print("Predictions:", model.predict(X))
```

```
Weights: [[0.2 0.2]]
Bias: [-0.2]
Predictions: [0 0 0 1]
```

The code shows a scikit-learn Perceptron implementation for the AND gate problem.

The code:

1. Imports NumPy, scikit-learn's Perceptron, and matplotlib
2. Sets up the training data for the AND gate
3. Initializes a Perceptron with 100 max iterations and a random seed of 42
4. Trains the perceptron on the AND gate data
5. Prints the learned weights, bias, and predictions

The output shows:

- **Weights: [[0.2 0.2]]** - The perceptron learned to assign a weight of 0.2 to both inputs
- **Bias: [-0.2]** - The bias is -0.2
- **Predictions: [0 0 0 1]** - The perceptron correctly classified all four examples of the AND gate

With these weights and bias, the decision function is: 0.2×(input1) + 0.2×(input2) – 0.2

For the four input combinations:

- [0,0]: 0.2×0 + 0.2×0 - 0.2 = -0.2 < 0 → output 0
- [0,1]: 0.2×0 + 0.2×1 – 0.2 = 0 → output 0
- [1,0]: 0.2×1 + 0.2×0 – 0.2 = 0 → output 0
- [1,1]: 0.2×1 + 0.2×1 – 0.2 = 0.2 > 0 → output 1

This perceptron implements the AND gate logic.

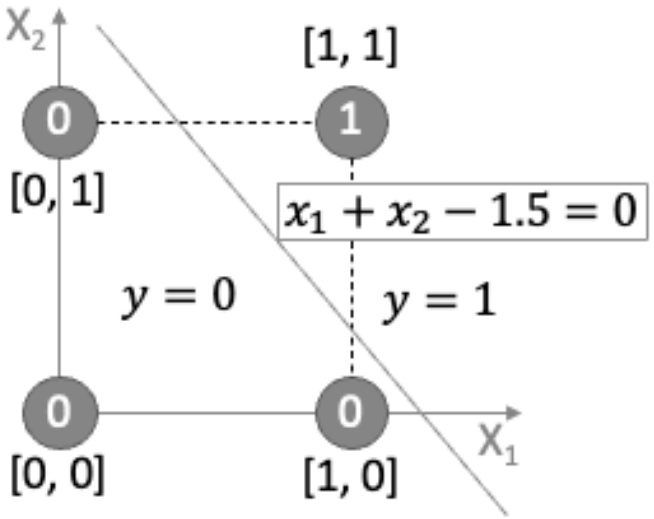
The decision boundary is the line $2x_1 + 2x_2 - 0.2 = 0$, which separates the point (1,1) from the other three points.

Open in Colab

Decision Boundary

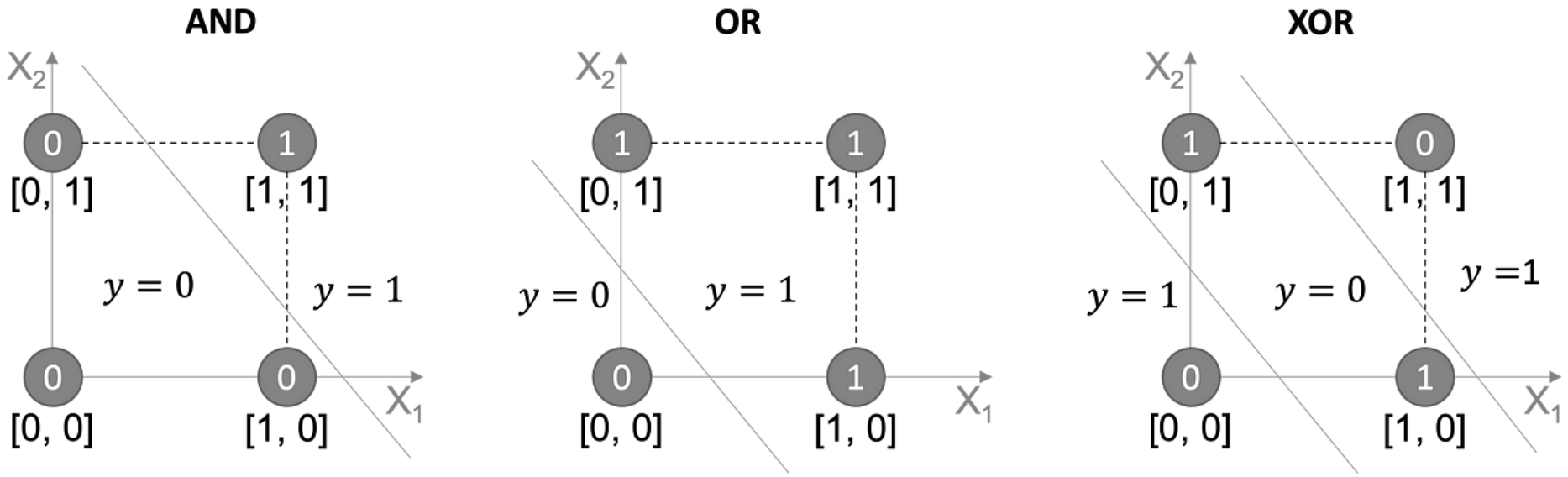
- The perceptron learns a decision boundary: $w_1x_1 + w_2x_2 + b = 0$
- Points above the line are classified as 1
- Points below the line are classified as 0
- For AND gate, only the point (1,1) should be above the line

X_1	X_2	y
0	0	0
0	1	0
1	0	0
1	1	1



Limitations of Simple Perceptron

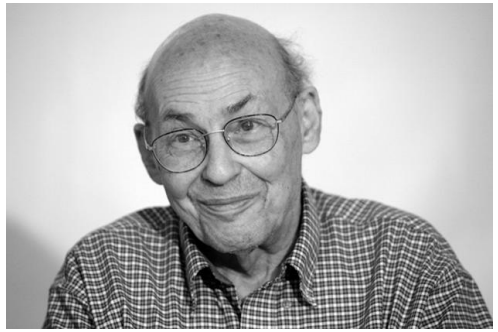
- Can only learn linearly separable patterns
- Cannot solve XOR problem (need multiple layers)
- No probabilistic output
- Simple update rule isn't suitable for complex problems



Introduction to Neural Networks

Multi-Layer Neural Networks

The Multi-Layer Percecptron (MLP)



Limitations of the Perceptron

While useful for linearly separable problems, the single perceptron cannot solve complex problems like XOR classification, as demonstrated by Minsky and Papert in their 1969 book "Perceptrons.“

The Multi-Layer Perceptron

The Multi-Layer Perceptron addresses the limitations of the single perceptron by introducing:

- Multiple layers of neurons
- Non-linear activation functions
- More sophisticated learning algorithms

Structure of an MLP

Definition: An MLP is a class of feedforward artificial neural network that consists of at least three layers of nodes: **input**, **hidden**, and **output** layers.

Key Feature: Each neuron in one layer is connected to every neuron in the next layer (fully connected).

1. Input Layer

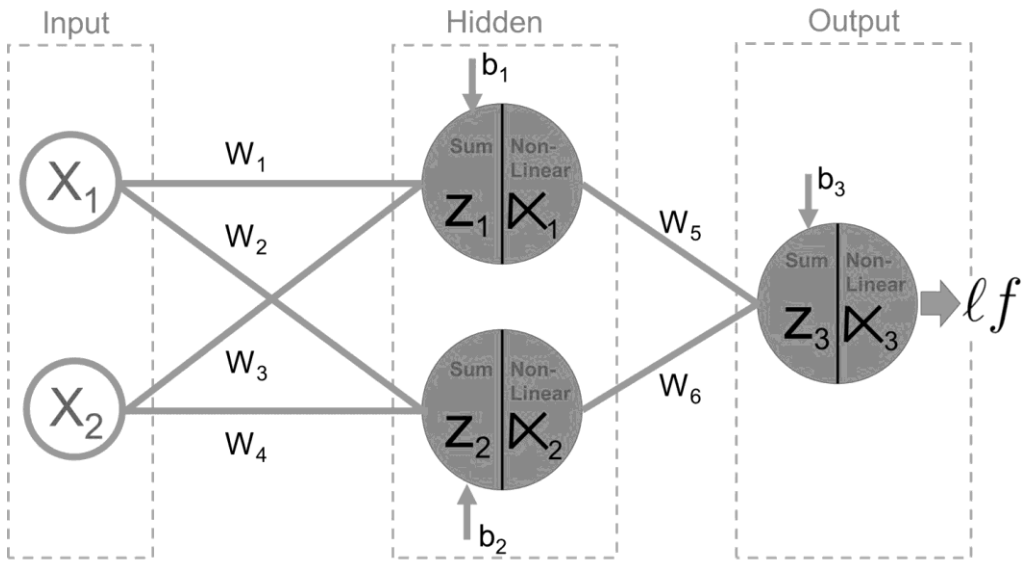
- Receives the raw input features
- One neuron per input feature
- No computation occurs here; inputs are simply passed forward

2. Hidden Layer(s)

- One or more layers between input and output
- Each neuron in a hidden layer:
- Receives inputs from all neurons in the previous layer
- Computes a weighted sum
- Applies a non-linear activation function
- Passes the result to the next layer

3. Output Layer

- Produces the final prediction or classification
- Structure depends on the task:
  - Regression: Often a single neuron with linear activation
  - Binary classification: One neuron with sigmoid activation
  - Multi-class classification: Multiple neurons (one per class) with softmax activation



Structure of an MLP

3. Neurons and Connections

- Each neuron computes a weighted sum of inputs and applies an activation function
- Fully connected between layers (dense connections)

4. Activation Functions

- Introduce non-linearity
- Examples: ReLU, Sigmoid, Tanh, Softmax

5. Loss Function

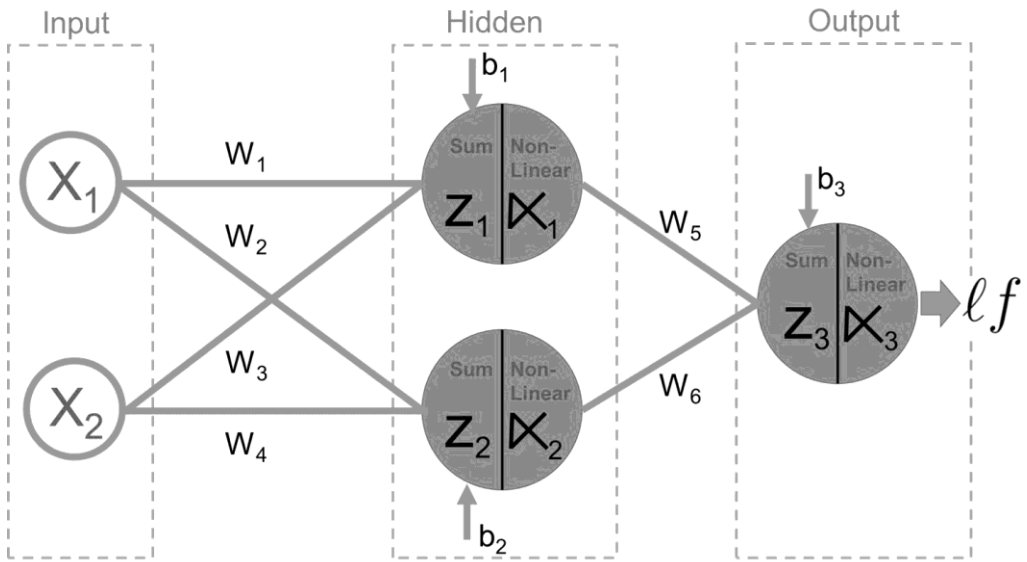
- Measures the error between predicted and true outputs
- Examples: Mean Squared Error, Cross-Entropy

6. Optimizer

- Updates weights to minimize loss
- Examples: SGD, Adam

7. Training Data

- Labeled data used to train the network
- Split into training, validation, and test sets



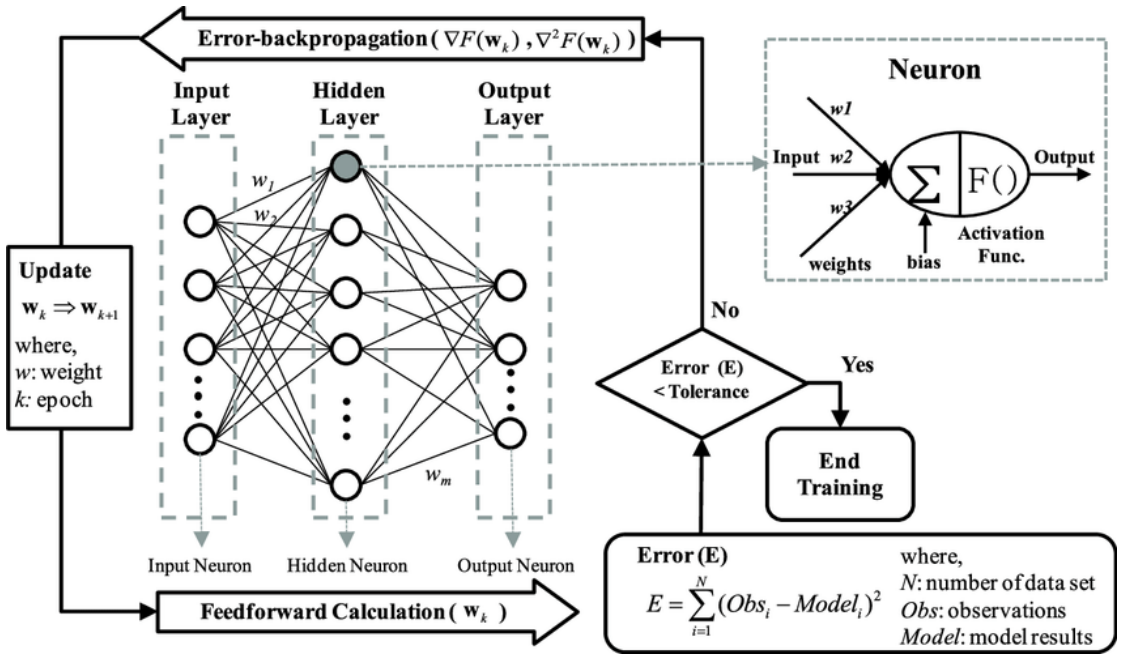
How an MLP Learn – Step by Step

Forward Propagation

- Input features pass through the network layer by layer.
- Each neuron computes a weighted sum of inputs and applies an activation function.
- The final layer produces a prediction.

Loss Computation

- A loss function measures the difference between predicted and actual outputs.
- Common loss functions:
  - Mean Squared Error (regression)
  - Cross-Entropy Loss (classification)



How an MLP Learn – Training Process

Backpropagation

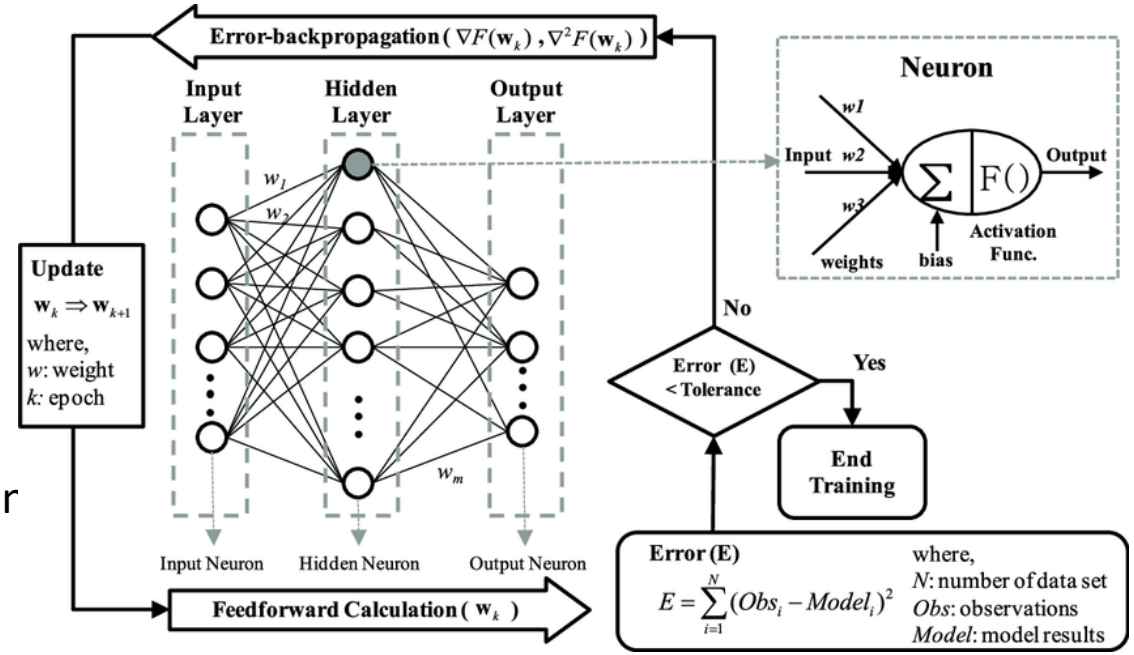
- Gradients of the loss are calculated with respect to each weight using the chain rule.
- This identifies how each weight contributed to the error.

Weight Update

- An optimizer (e.g., SGD) adjusts weights to reduce loss:
  - w := w - learning_rate × gradient
- This process repeats over multiple iterations (epochs) usir training data.

Goal

- Gradually minimize the loss and improve prediction accuracy.
- **Loss Function**: MSE for regression, Cross-Entropy for classification.
- **Optimization**: Backpropagation + Gradient Descent (or Adam).



Activation Functions other than Step Function

- Neural Networks use activation functions other than the simple step function in the Perceptron.
- Activation Function helps the neural network use important information while suppressing irrelevant data points (i.e., allows local “gating” of information).

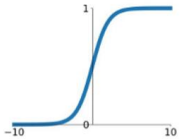
Common Activation Functions: A Toolbox for Neural Networks

- **Sigmoid**: Compresses input to a range between 0 and 1.
- **Tanh**: Like sigmoid but ranges from -1 to 1.
- **ReLU**: Passes positive values, zeroes out negatives.
- **Leaky ReLU**: Allows a small, non-zero gradient for negative inputs.
- **Maxout**: Chooses the most active linear response.
- **ELU**: Smoothly transitions through zero and allows negative outputs.

Each function offers a trade-off between simplicity, flexibility, and computational cost—choose based on the task and depth of your model

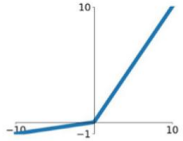
Sigmoid

$\sigma(x) = \frac{1}{1+e^{-x}}$



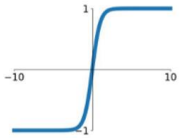
Leaky ReLU

$\max(0.1x, x)$



tanh

$\tanh(x)$

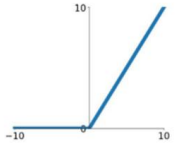


Maxout

$\max(w_1^T x + b_1, w_2^T x + b_2)$

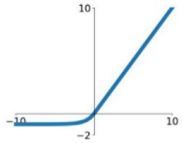
ReLU

$\max(0, x)$



ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



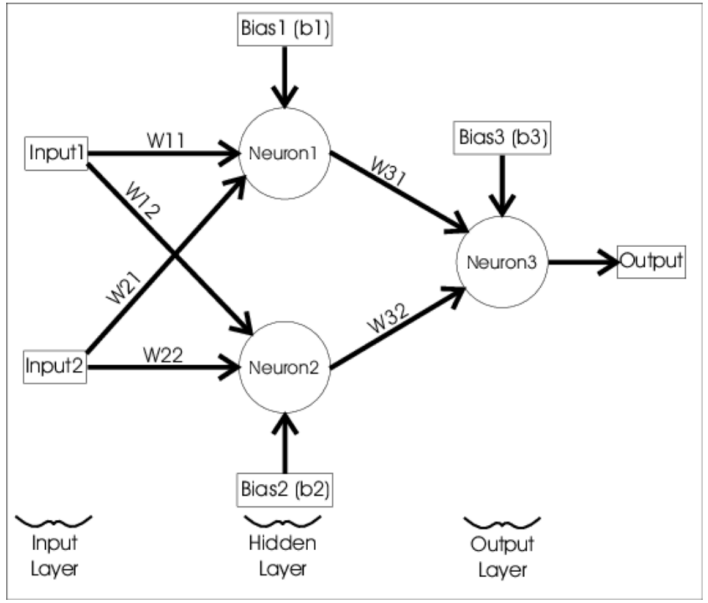
https://ml-explained.com/blog/activation-functions-explained

Multi-Layer Perceptron in Python

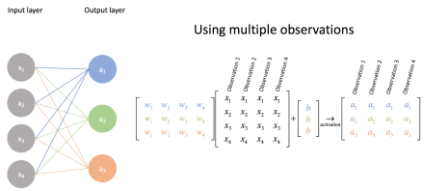
```python
from sklearn.neural_network import MLPClassifier
import numpy as np
# XOR input and output
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([0, 1, 1, 0])
# Define MLP with 1 hidden layer of 2 neurons (minimal config for XOR)
mlp = MLPClassifier(hidden_layer_sizes=(2,), activation='tanh’, solver='adam', learning_rate_init=0.01,
    max_iter=10000, random_state=42)
# Train the model
mlp.fit(X, y)
# Make predictions
predictions = mlp.predict(X)
print("Predictions:\n", predictions)
print("\nWeights (input to hidden):\n", mlp.coefs_[0])
print("\nBias hidden:\n", mlp.intercepts_[0])
print("\nWeights (hidden to output):\n", mlp.coefs_[1])
print("\nBias output:\n", mlp.intercepts_[1])
```

```
Weights (input to hidden):
 [[ 2.7144501   3.27401218]
 [-2.73418453 -3.17014048]]

Bias hidden:
 [ 1.21994174 -1.63451199]
```

```
Weights (hidden to output):
 [[-4.37775211]
 [ 4.46553876]]

Bias output:
 [3.61855675]
```



The Neural Network Model to solve the XOR Logic (from: https://stopsmokingaids.me/)



Open in Colab

The Difference Between a Perceptron and an MLP?

Feature	Perceptron	Multilayer Perceptron (MLP)
Layers	Only 1 layer (no hidden layers)	2+ layers (has hidden layers)
Activation Function	Step function (hard threshold)	Nonlinear (e.g., ReLU, sigmoid, tanh)
Learning Rule	Simple rule: update on error	Gradient descent + backpropagation
Tasks It Can Solve	Only linearly separable problems	Nonlinear, complex problems

Introduction to Word Embeddings

Converting Words to Numbers

Old Method:- One-Hot Encoding

Each word is represented by setting one dimension to 1 and all the other dimensions to ZEROS.

id	color
1	red
2	blue
3	green
4	blue

One Hot Encoding →

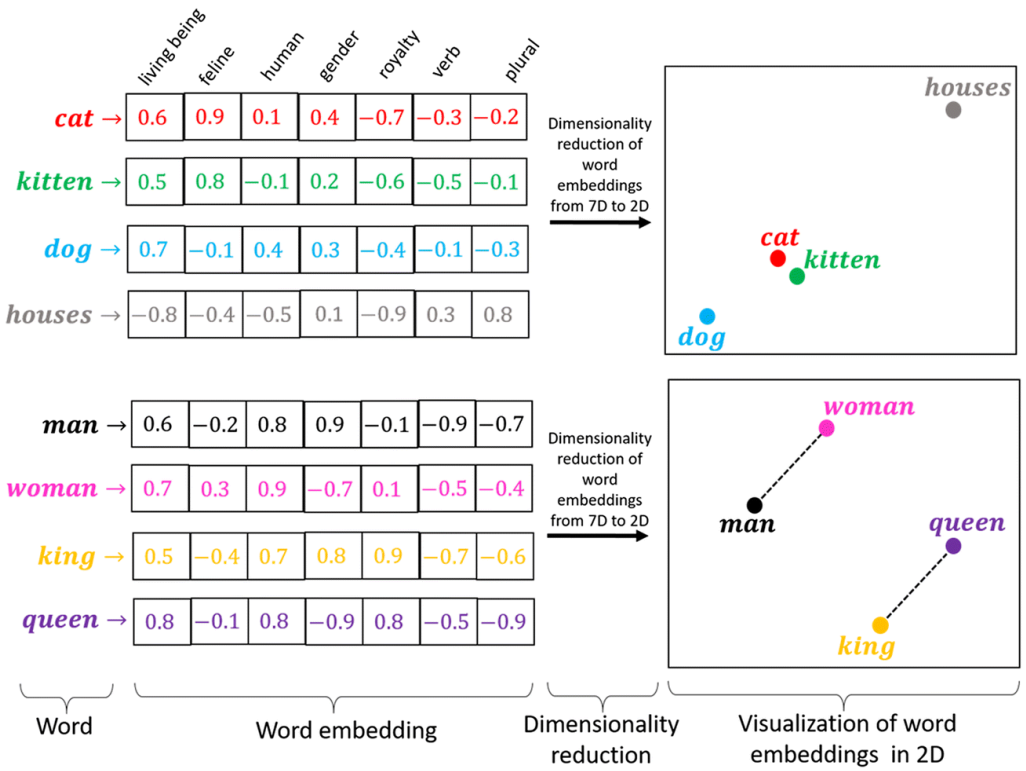
id	color_red	color_blue	color_green
1	1	0	0
2	0	1	0
3	0	0	1
4	0	1	0

Limitations:

- Vocabulary of 100,000 words → 100,000 dimensions [0010000000000000000000000000000000000000...etc.]
- All words equally distant from each other
- No semantic meaning captured

Word Embeddings and Vector Spaces

- Words are encoded as dense numerical vectors instead of one-hot or sparse representations.
- Similar words → similar vectors
- Typically, 50-300 dimensions (vs. vocabulary size in 1-hot-encoding)
- Word Embeddings captures semantic and syntactic relationships about/between words.
- Each dimension potentially captures some syntactic or semantic meaning.
- These vectors are learned from text by models like Word2Vec.

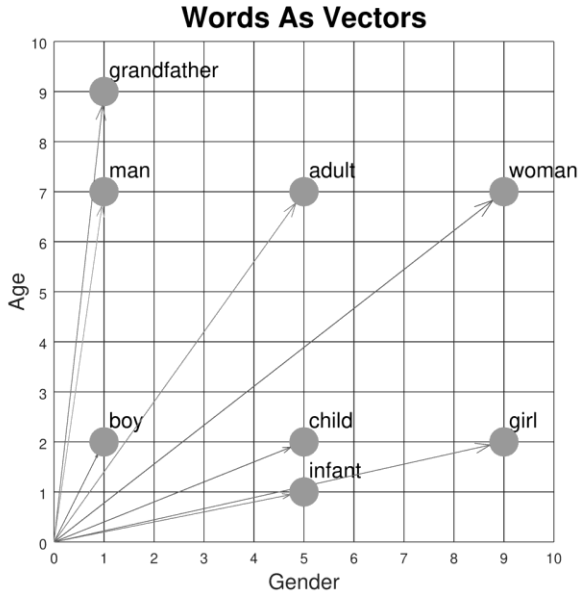


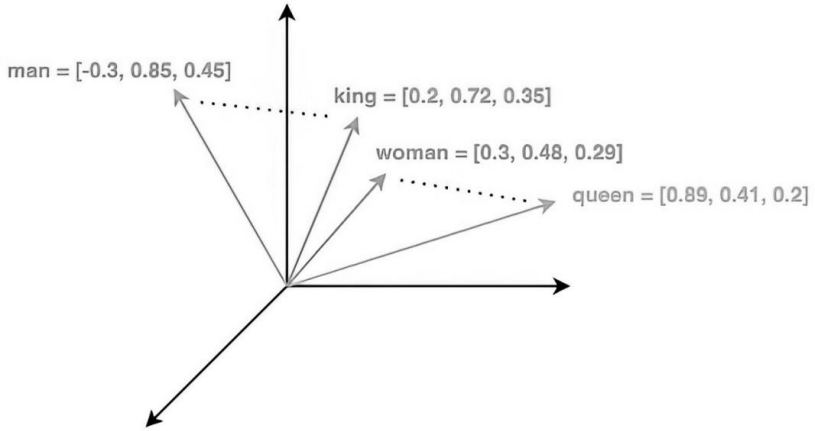
Vector Spaces and Word Embeddings

- A vector space is a mathematical space where each word is represented as a point (or vector) in multi-dimensional space.
- Makes it possible to compare, visualize, and manipulate meanings of words using math.
- Enables operations like:
  - Similarity: "king" is close to "queen"
  - Analogy: "king" - "man" + "woman" ≈ "queen"

Properties of Vector Space

- Semantic relationships are preserved (e.g., “man" is closer to “grandfather" than “girl").
- Closer Vectors → Similar Meanings
- Vectors Farther Apart → Dissimilar Meanings





Key Intuition

- Distributional hypothesis - "You shall know a word by the company it keeps" (J.R. Firth, 1957)
- Words in similar contexts have similar meanings:
- "The **cat** sat on the mat"
- "The **dog** sat on the mat"
- "The **rabbit** sat on the mat"
- → cat, dog, rabbit learn similar representations

Word2Vec (Tomas Mikolov et al., 2013)

Word2Vec was developed by Tomas Mikolov and team at Google.

Mikolov presented two architectures:

1) CBOW (Continuous Bag of Words): Predict word from context

- Input: [the, ___, sat, on] → Output: cat

2) Skip-gram: Predict context from word

- Input: cat → Output: [the, sat, on, mat]

- Trained on massive text corpora using simple neural networks

Mikolov Approach – Convert Text into Input-Output Pairs

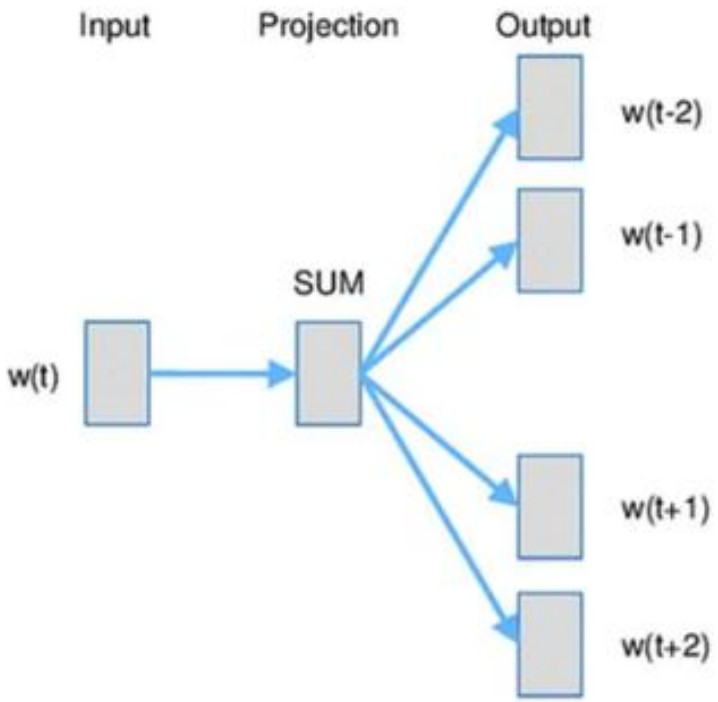
Suppose we have these sentences (corpus), we can convert them into input output pairs to be used for training a neural network:

Large language models are transforming business applications by introducing new levels of automation, intelligence, and personalization across industries. These models, trained on vast amounts of text data, can understand and generate human-like language, allowing businesses to enhance customer interactions, streamline operations, and gain deeper insights from data. In customer service, for instance, large language models power advanced chatbots and virtual assistants that can handle complex queries, provide 24/7 support, and adapt to customer needs in real time. In marketing, they help craft personalized messages, analyze consumer sentiment, and optimize content strategies with remarkable precision. Organizations are also using these models to automate report generation, summarize large documents, and even assist in writing code or drafting legal documents, significantly reducing time and cost. Moreover, they enable better decision-making by turning unstructured data—such as emails, reviews, or social media posts—into actionable business intelligence. As companies integrate these tools, they not only boost productivity but also redefine how humans and machines collaborate in the workplace. However, this transformation also raises questions about data privacy, ethical AI use, and workforce adaptation, making responsible deployment as important as technological innovation itself.

Skip-gram Architecture

- **Source Sentence:** "Large language models are transforming business applications"
- **Window Size:** 2 (2 words on each side of target word)
- **Task:** Predict context words from target word
- Given Target Word Predict → Context Words

Input (Target Word)	Output (Context Words)
Large	[language, models]
language	[Large, models, are]
models	[Large, language, are, transforming]
are	[language, models, transforming, business]
transforming	[models, are, business, applications]
business	[are, transforming, applications]
applications	[transforming, business]

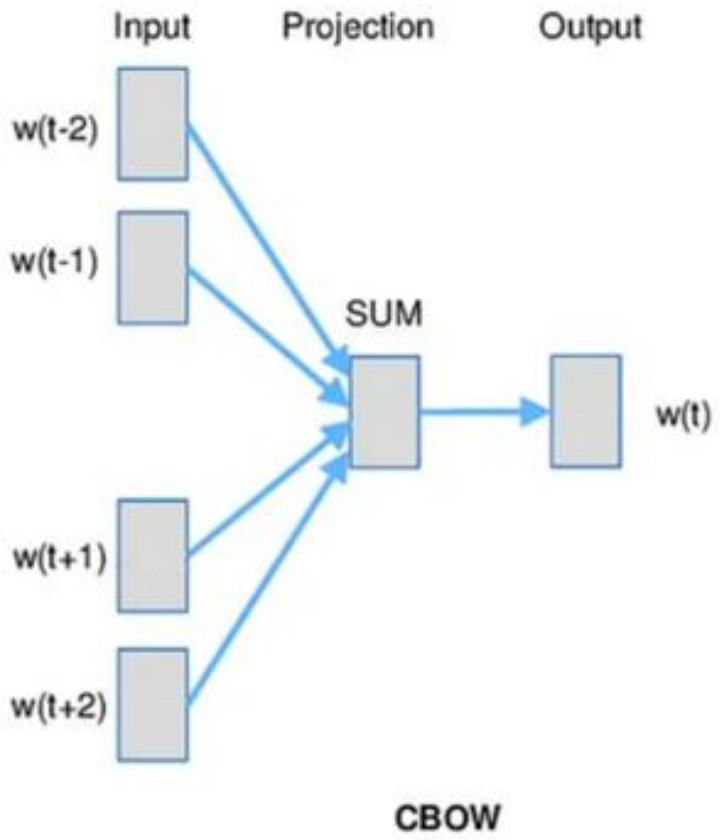


Skip-gram

CBOW (Continuous Bag of Words) Architecture

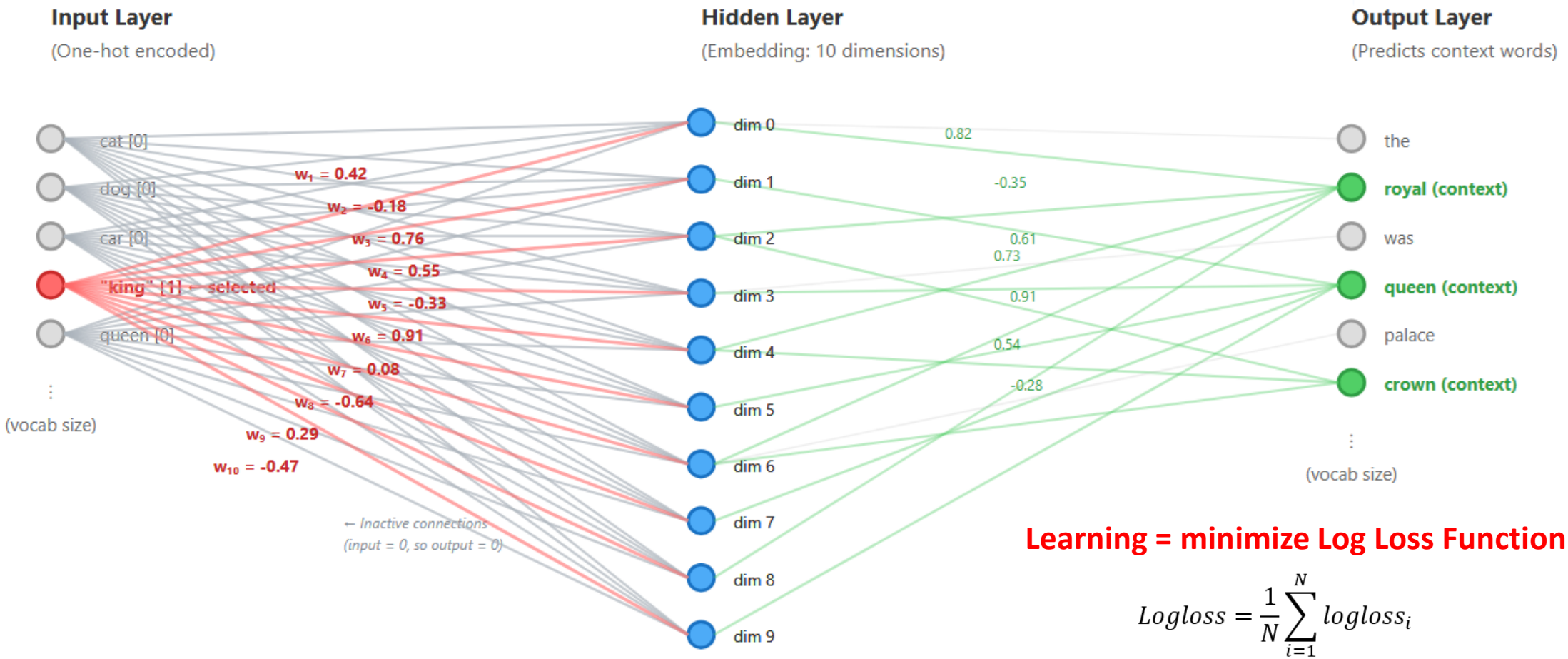
- **Source Sentence:** "Large language models are transforming business applications"
- **Task:** Predict target word from context words
- Given Context Words Predict → Target Word

Input (Context Words)	Output (Target Word)
[language, models]	Large
[Large, models, are]	language
[Large, language, are, transforming]	models
[language, models, transforming, business]	are
[models, are, business, applications]	transforming
[are, transforming, applications]	business
[transforming, business]	applications



Word2Vec Skip-gram: Learning Word Embeddings

Training sentence: "The **king** wore a royal crown, and the queen stood beside him"



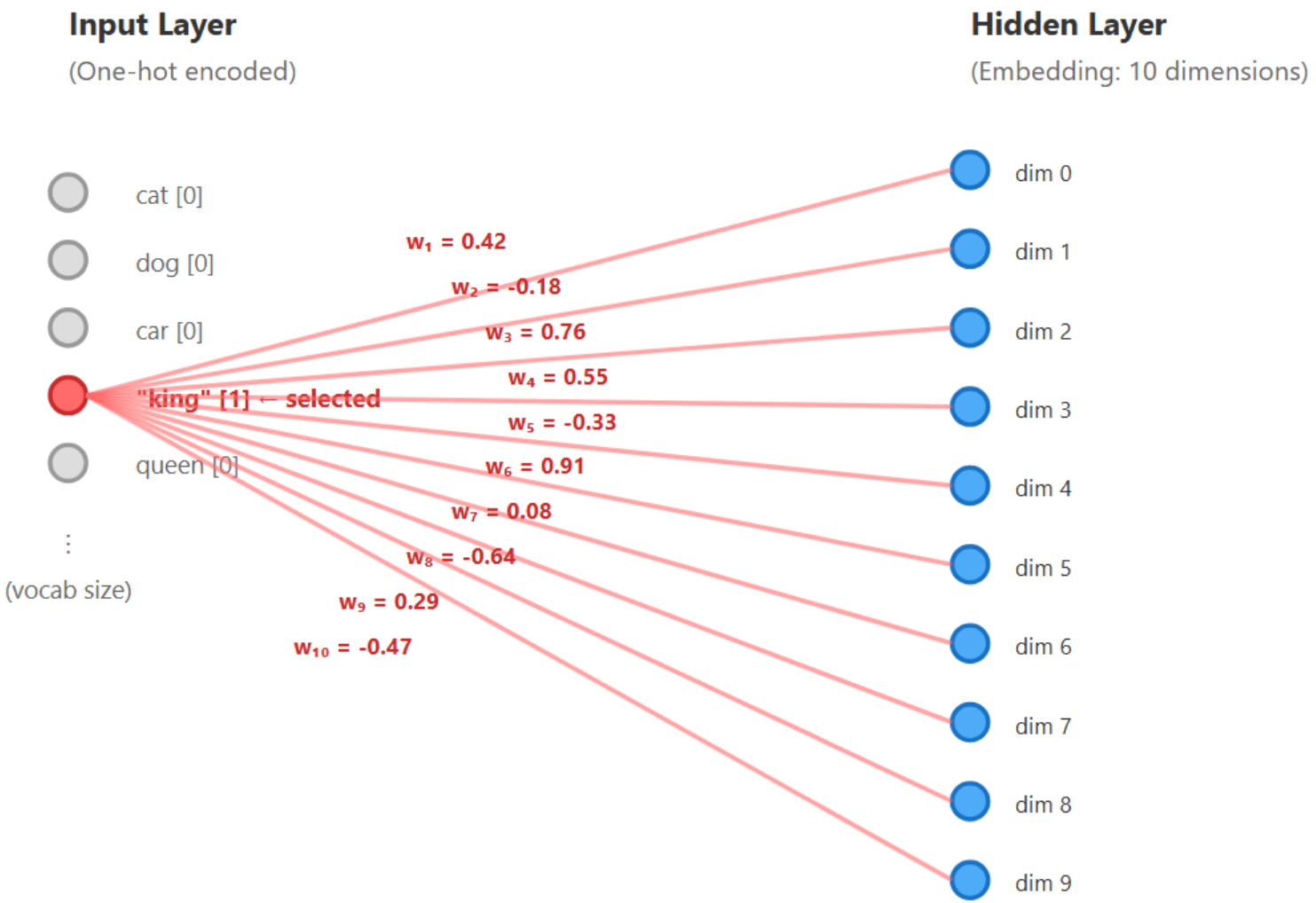
Learning = minimize Log Loss Function

$$Logloss = \frac{1}{N}\sum_{i=1}^{N} logloss_i$$

Training Process: Input Word "king" → Embedding → Predict Context Words

- Input→Hidden weights = Word Embeddings (e.g., king = [0.42, -0.18, 0.76, 0.55, -0.33, 0.91, 0.08, -0.64, 0.29, -0.47])
- Hidden→Output weights predict context: Given "king", maximize probability of "royal", "queen", "crown"
- Backpropagation updates both weight matrices → words in similar contexts get similar embeddings

Skip-Gram Model



Word2Vec Skip-gram: What's Really Happening?

The Story

We want to learn word vectors where similar words have similar vectors. The key insight: **words that appear in similar contexts are similar**. So we set up a prediction task.

For every word pair (focus word w, context word c) in our text, we ask: *how probable is it to see context word c near focus word w?*

$p(c|w) = \frac{\exp(v_c \cdot v_w)}{\sum_{c'} \exp(v_{c'} \cdot v_w)}$

The dot product $v_c \cdot v_w$ measures similarity. We use softmax to turn these scores into probabilities. Words that actually appear together should get high probability.

The Learning Process

We **maximize** the likelihood of all the (word, context) pairs we observe in our training text:

$\max_\theta \sum_{(w,c)\in D} \log p(c|w)$

This is softmax regression where we're classifying: "given focus word w, which context word c will appear?"

In practice, we flip this to **minimize the negative log likelihood** (standard loss function):

$\mathcal{L} = -\sum_{(w,c)\in D} \log p(c|w)$

As we optimize, word vectors adjust so that words appearing in similar contexts naturally end up close together in vector space.

Challenge: The denominator sums over the entire vocabulary (expensive!)
Solution: Negative sampling or hierarchical softmax

Word2Vec Skip-gram Model

Institute of Analytics Programme

University of Petra

Goal

Learn word vector representations by predicting **context words** from a **focus word**

The Probability Model

$$p(c|w;\theta) = \frac{\exp(v_c \cdot v_w)}{\sum_{c' \in C} \exp(v_{c'} \cdot v_w)}$$

- **Numerator**: Dot product $v_c \cdot v_w$ measures similarity between word vectors
- **Denominator**: Softmax normalization over all possible context words
- **Parameters** θ: All word vectors we're learning

The Optimization Problem

Maximize log likelihood over training data D:

$$\max_\theta \sum_{(w,c)\in D} \log p(c|w) = \sum_{(w,c)\in D} \left[v_c \cdot v_w - \log \sum_{c'} \exp(v_{c'} \cdot v_w) \right]$$

In practice: Minimize the **negative log likelihood loss**

$$\mathcal{L} = -\sum_{(w,c)\in D} \log p(c|w)$$

Key Insights

✓ This is **softmax regression** (multi-class classification where classes = all vocabulary words)

✓ Taking log turns products → sums, prevents underflow

✓ **Computational challenge**: Denominator requires summing over entire vocabulary for each prediction

✓ **Solution**: Negative sampling or hierarchical softmax

NOT REQUIRED

Measuring Similarity Between Word Vectors

Why Compare Word Vectors?

- Word embeddings map words into a vector space.
- **Words with similar meanings** are placed **close together** in that space.
- To quantify this "closeness," we use **vector similarity**.

Cosine Similarity

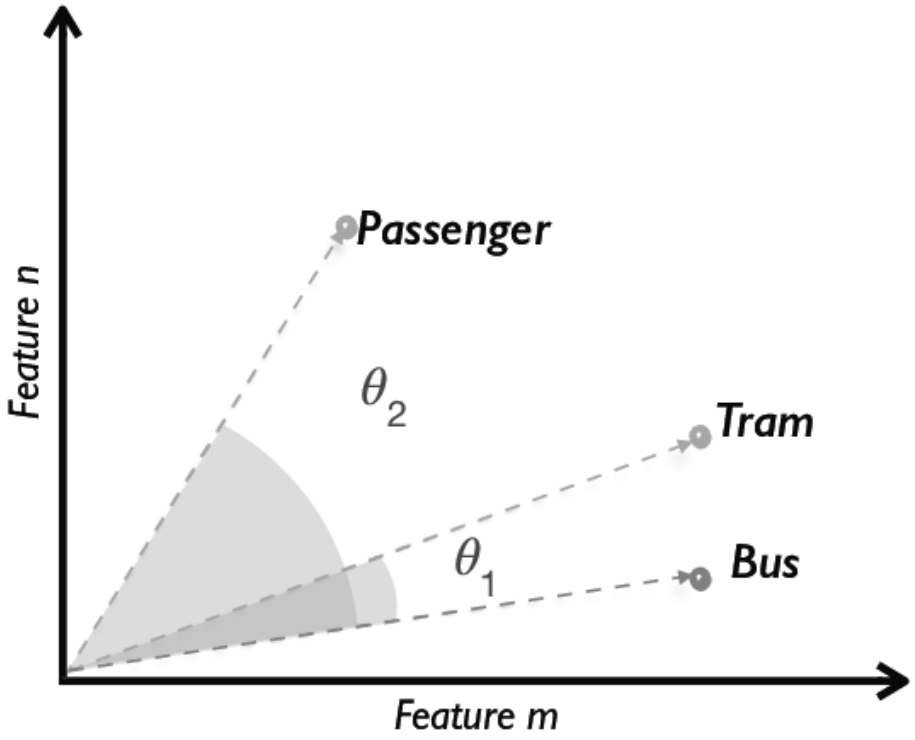
Most common metric used to compare word vectors:

$$\text{cosine_similarity}(\vec{A}, \vec{B}) = \frac{\vec{A} \cdot \vec{B}}{\|\vec{A}\| \|\vec{B}\|}$$

- Measures the **angle** between two vectors (not their magnitude).
- Ranges from **-1 to 1**:
  - 1 → Same direction (very similar)
  - 0 → Orthogonal (unrelated)
  - -1 → Opposite directions (very different)

Intuition

- Vectors for `"king"` and `"queen"` will have high cosine similarity.
- Vectors for `"apple"` and `"keyboard"` will have low similarity.



Use Pre-Trained Embeddings

Gensim is an open-source Python library used for **topic modeling** and **natural language processing (NLP)**.
It's great for working with **large text datasets** because it doesn't need to load all the data into memory at once.

Key features:

- Builds and uses **word embeddings** (like Word2Vec, FastText, Doc2Vec).
- Measures **semantic similarity** between words or documents.
- **Efficient and memory-friendly**, ideal for handling big collections of text.

```python
import gensim.downloader as api
from gensim.models import Word2Vec

# Load pre-trained Word2Vec model
word2vec_model = api.load("word2vec-google-news-300")

# Find similar words
similar_words = word2vec_model.most_similar('computer', topn=5)
print("Words similar to 'computer':", similar_words)

# Word analogies
result = word2vec_model.most_similar(positive=['woman', 'king'],
    negative=['man'], topn=1)
print("king - man + woman =", result)

# Train your own Word2Vec model
sentences = [["cat", "say", "meow"], ["dog", "say", "woof"]]
model = Word2Vec(sentences, vector_size=100, window=5, min_count=1,
    workers=4)

# Get vector for a word
cat_vector = model.wv['cat']
print("Vector for 'cat':", cat_vector[:5])  # Show first 5 dimensions
```

Open in Colab

Applications of Word Embeddings in NLP

1. **Semantic Similarity**
   Measure how similar two words, phrases, or documents are by comparing their vector representations.
   Example: Identifying that "doctor" and "physician" are closely related.

2. **Text Classification**
   Used as input features for tasks like spam detection, sentiment analysis, and topic classification.
   Embeddings provide rich, dense input for machine learning models.

3. **Named Entity Recognition (NER)**
   Help identify proper nouns and classify them into categories like person, location, or organization.
   Embedding-based models improve contextual understanding of named entities.

4. **Machine Translation**
   Map words from one language to another by aligning embeddings in multilingual space.
   Improves translation accuracy by leveraging semantic proximity.

5. **Question Answering & Chatbots**
   Used to understand queries and match them with appropriate answers or responses.
   Enable bots to interpret intent and context more accurately.

- **6. Information Retrieval**
  Enhance search engines by retrieving results based on semantic meaning, not just keyword matches.
  Example: Searching for “heart attack” returns documents containing “cardiac arrest.”